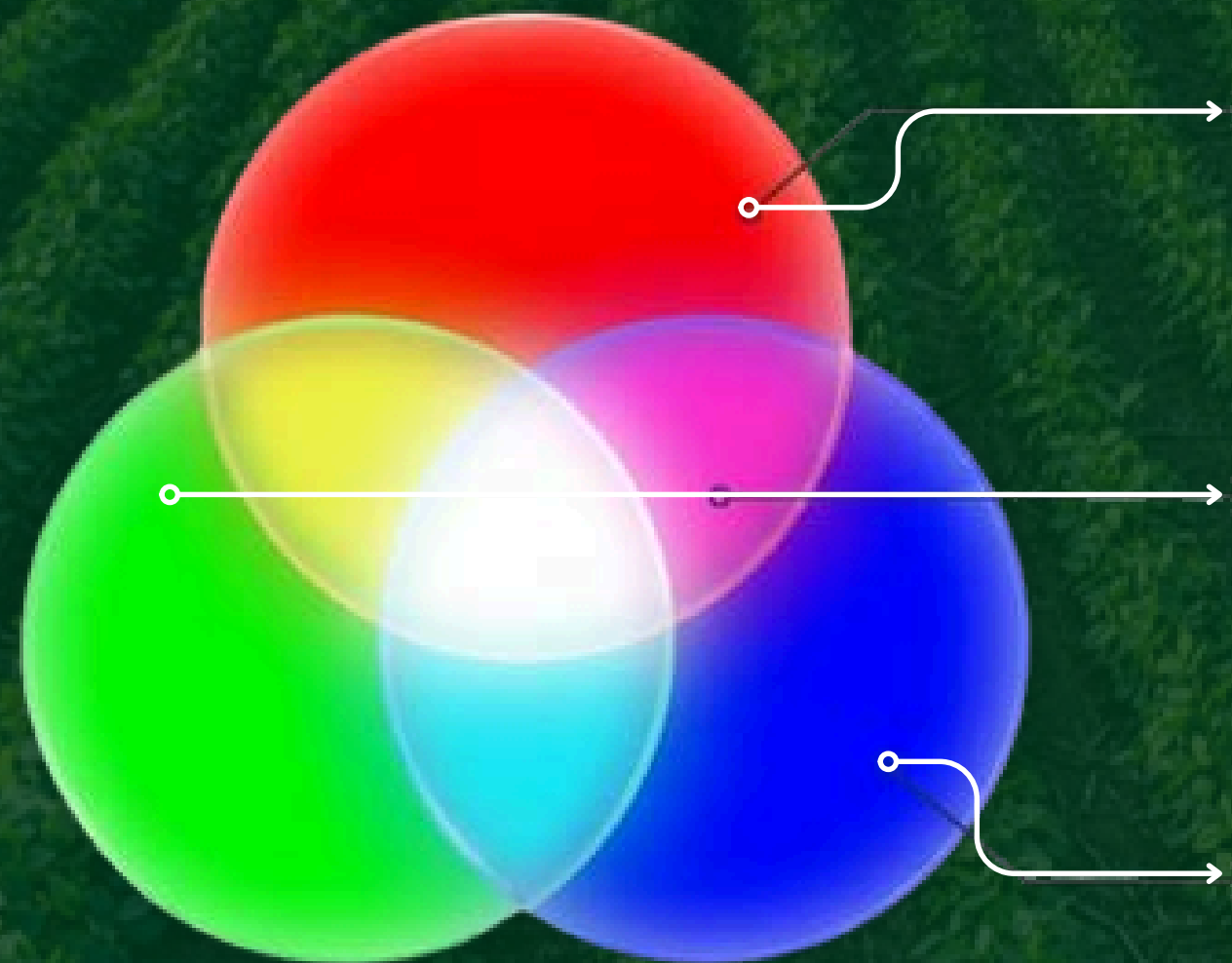


20	19	20	200	202	201
21	20	19	205	198	207
19	20	21	199	206	199
20	21	20	210	200	205
21	19	20	201	209	202
20	20	19	208	203	210

การคัดแยกขอบ (Edge Detection)

คอมพิวเตอร์หาขอบผลไม้ได้อย่างไร?
ระบบจะเช็คจุดที่ตัวเลข "เปลี่ยนกะทันหัน"
เช่น จาก 20 กระโดดไป 200 แสดงว่านี่คือ
เส้นแบ่งระหว่าง "ฉากหลัง" กับ "ผลไม้"

ภาพสี RGB : หลอดไฟ 3 ดวง ใน 1 พิกเซล



Red (สีแดง)

Green (สีเขียว)

Blue (สีน้ำเงิน)

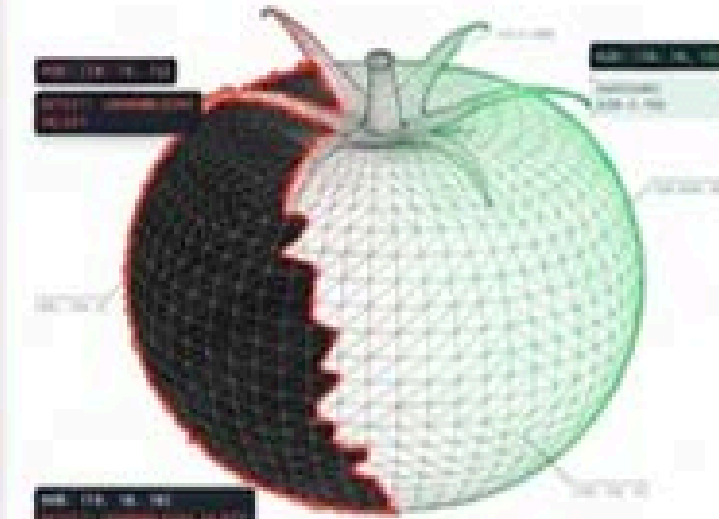
ทำไมใช้ค่าสี RGB อย่างเดียวยังไม่พอ?



แสงตกกระทบบสม่ำเสมอ ->
RGB อ่านค่าเป็น 'สีแดง'



ปัญหาเรื่องแสงเงา:
แสงธรรมชาติเปลี่ยนทิศทาง



Error: สีดำ?

สรุป เมื่อแสงตกกระทบบผลไม้ไม่เท่ากัน ค่าตัวเลข RGB จะเปลี่ยนไป
ทำให้คอมพิวเตอร์สับสนและเข้าใจผิดว่าส่วนที่เป็นเงาคือรอยตำหนิหรือสีดำ ไม่ใช่สีแดง

โหมด HSV : ทางออกเพื่อทะลุข้อจำกัดเรื่องเงา

โหมด RGB

วัดค่าสีรวมกับความสว่าง
(โดนเงาหลอกได้ง่าย)

โหมด HSV

แยกมิติสีออกจากแสง

- H (Hue): ชนิดของสี (เขียว ส้ม เหลือง แดง)
- S (Saturation): ความสดของสี
- V (Value): ความสว่าง

ประโยชน์สูงสุด: ระบบจะมองเฉพาะค่า H (ชนิดสี) เพื่อเช็คความสุก
ทำให้สามารถอ่านค่าสีผลไม้ได้ถูกต้องแม้จะมีเงาตกกระทบบ



ตรรกะการตั้งเกณฑ์ (Threshold)

คัดแยกความสุกของมะเขือเทศ



การควบคุมสภาพแวดล้อมหน้าจอ : คุณแจ้สู่ความแม่นยำ 100%

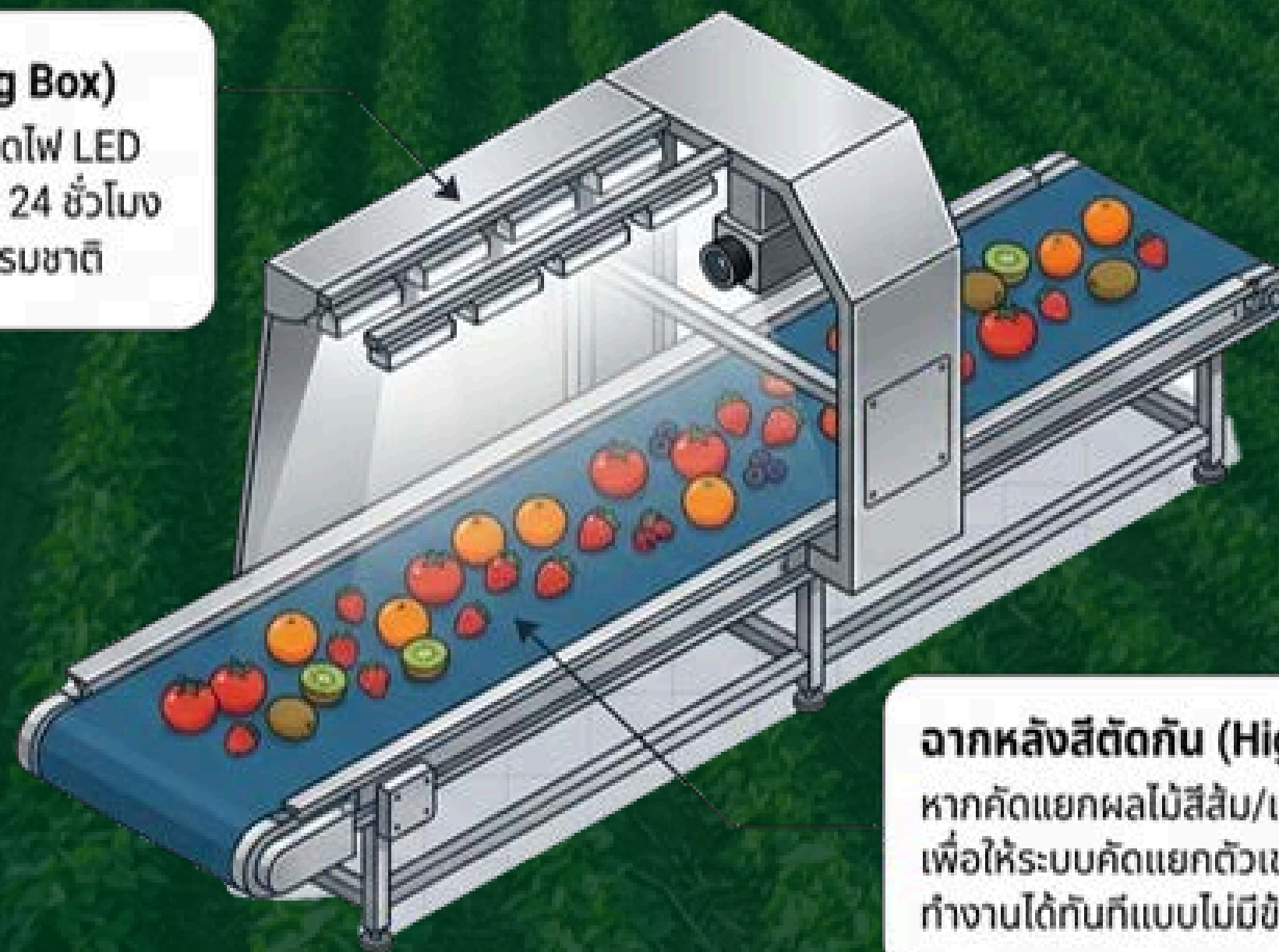


ซอฟต์แวร์ที่ฉลาดที่สุดก็ล้มเหลวได้หากแสงภายนอก เปลี่ยน
กฎเหล็กของการทำ Computer Vision คือ ห้ามใช้แสงธรรมชาติโดยเด็ดขาด
แสงแดดเข้า-บ่ายที่ไม่เท่ากัน หรือเมฆที่บังแสงอาทิตย์ จะทำให้ค่าสีในภาพเพี้ยนและระบบทำงานผิดพลาดทันที

โครงสร้างของระบบคัดแยกที่สมบูรณ์แบบ

ตู้คุมแสง (Lighting Box)

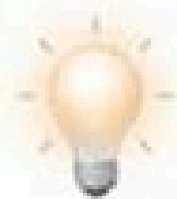
ครอบกล่องและติดตั้งหลอดไฟ LED
ที่มีความสว่างคงที่ตลอด 24 ชั่วโมง
เพื่อตัดปัญหาแสงธรรมชาติ



ฉากหลังสีตัดกัน (High-Contrast Background)

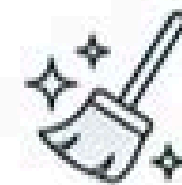
หากคัดแยกผลไม้สีสับ/แดง ต้องใช้สายพานสีน้ำเงินเข้มเข้ม
เพื่อให้ระบบคัดแยกตัวขอบผลผลิต (Edge Detection)
ทำงานได้ทันทีแบบไม่มีข้อผิดพลาด

Checklist 4 ข้อ : ถ่ายภาพผลผลิตอย่างไรไม่ให้ AI ลับสน



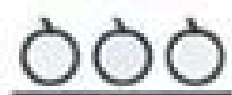
1. แสงสว่างสม่ำเสมอ

ต้องไม่มีเงาสาตกบบนผลผลิต และไม่มีจุดสะท้อนแสงจ้า (Glare) จากพื้นน้ำหรือความมัน



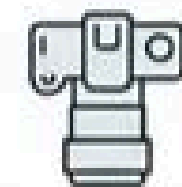
2. ฉากหลังสะอาด

ต้องไม่มีเศษขยะ ใบไม้ หรือวัตถุอื่นวางปะปนบนสายพาน



3. อย่างวางซ้อนกัน

วางผลผลิตแยกห่างกันเล็กน้อยเพื่อให้คอมพิวเตอร์นับจำนวนและแยกขอบได้ถูกต้อง



4. ตำแหน่งกล้องตั้งตรง

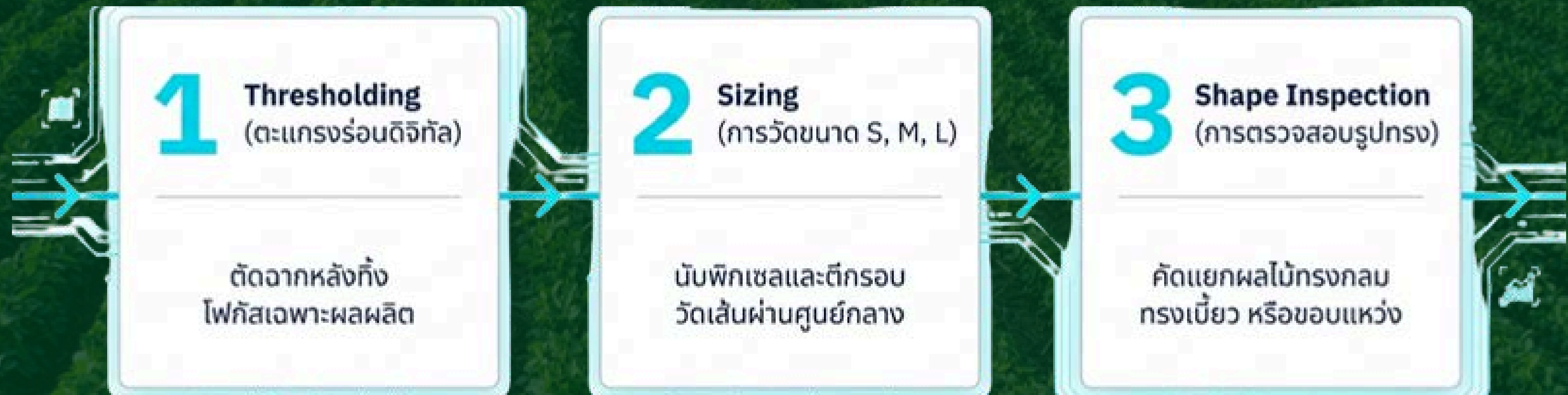
กล้องต้องทำมุมตั้งฉาก 90 องศา กับสายพาน เพื่อไม่ให้สัดส่วนและขนาดของผลไม้บิดเบี้ยว

สรุป Session 1 : จากการคุมแสงหน้างานสู่ตารางตัวเลขดิจิทัล

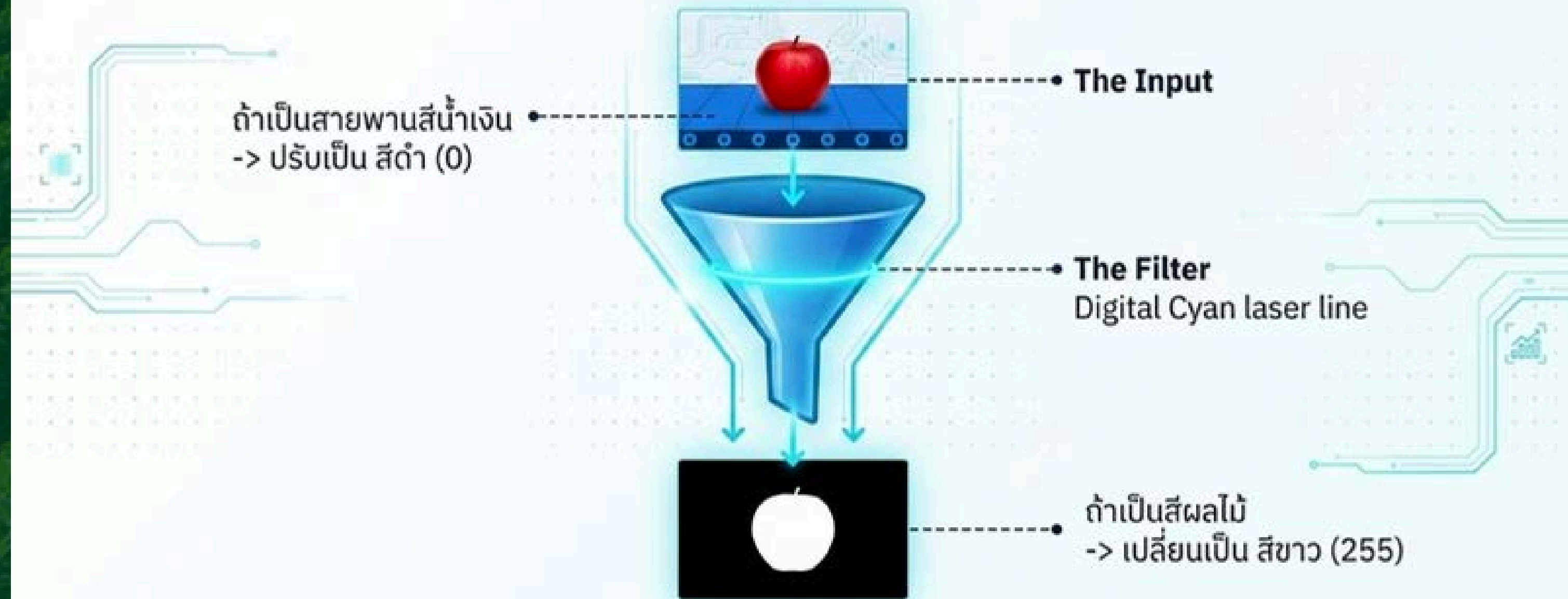


พัก 10 นาที ก่อนเริ่ม Session2 :
การวัดขนาด คัดไซส์ และตรวจสอบรูปทรง

โครงสร้างการทำงาน: จากภาพถ่ายสู่ข้อมูลเชิงตัวเลข (The 3-Step Pipeline)



ขั้นตอนที่ 1: การสร้าง "ตะแกรงร่อนดิจิทัล" (Thresholding) ตัดส่วนไม่สำคัญทิ้ง (Background) เพื่อให้คอมพิวเตอร์ทำงานได้แม่นยำ 100%



Binary Mask : แปลงภาพเป็น 2 สี (ขาว-ดำ)

The Physical



ความซับซ้อนสูง
(แสง, เงา, ริ้วรอย)

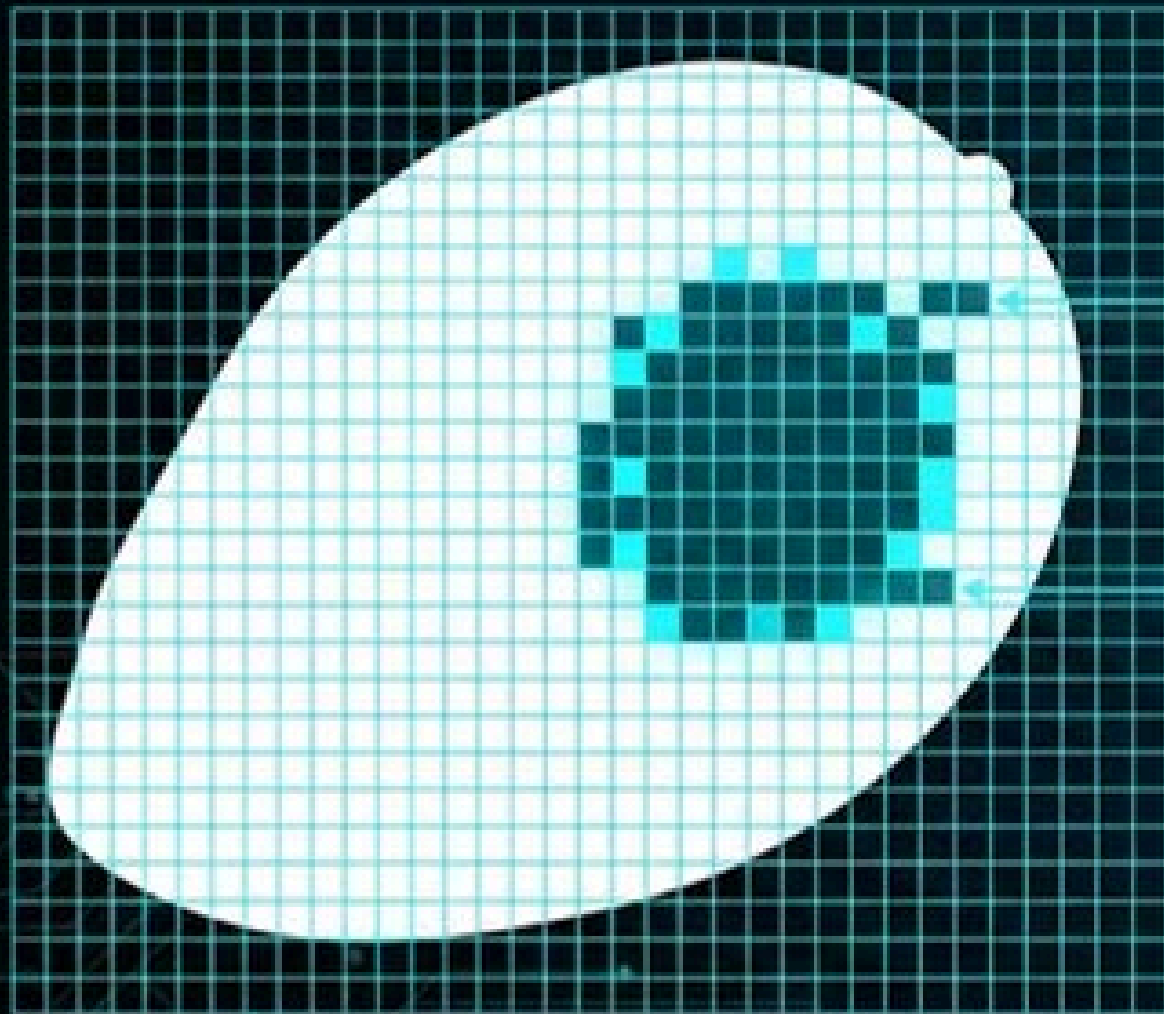
The Digital



เงาสีขาวเด่นชัดเจน
ลดความซับซ้อน
คอมพิวเตอร์พร้อมนับพื้นที่

ขั้นตอนที่ 2: นับพิกเซล = วัดพื้นที่ผลผลิต (Area)

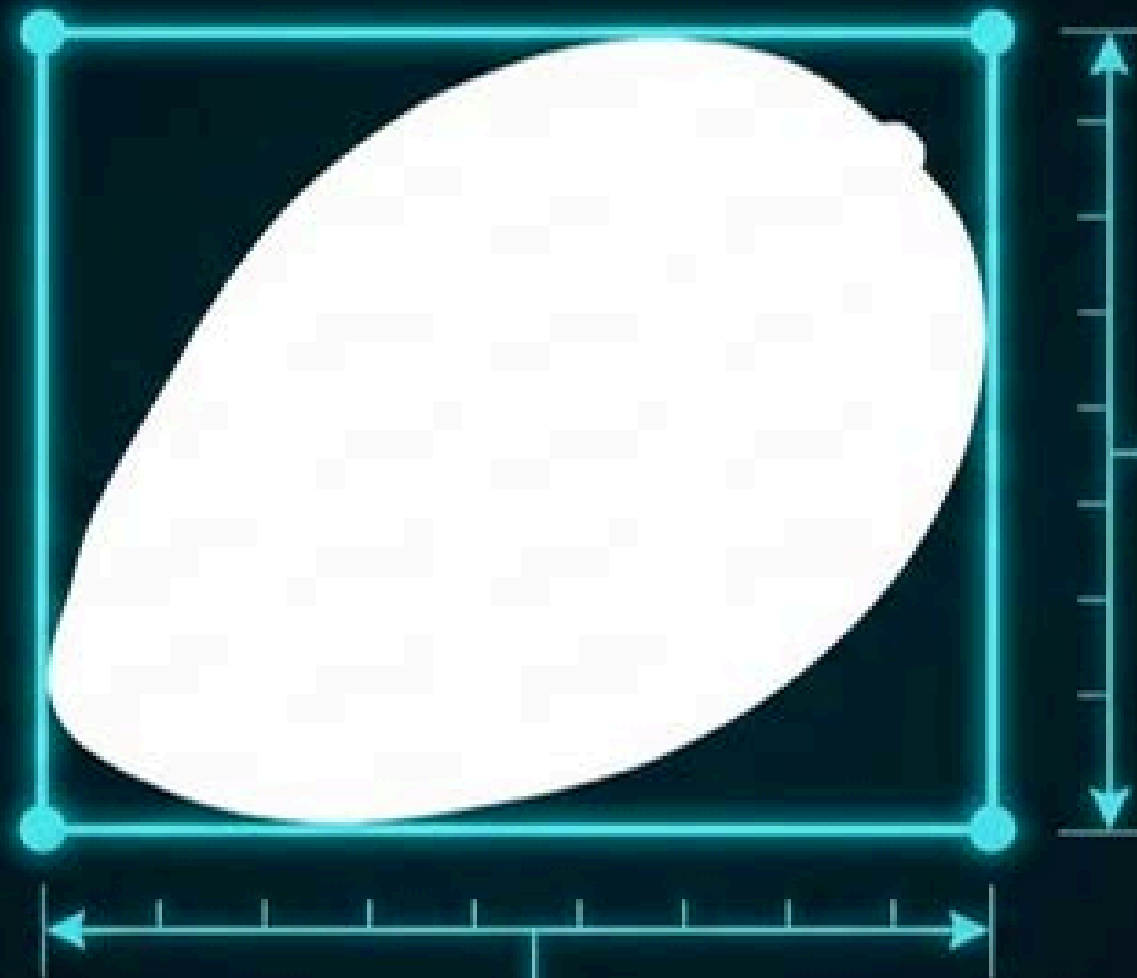
เมื่อเปลี่ยนเป้าหมายเป็นพื้นที่สีขาว คอมพิวเตอร์สามารถนับจำนวนช่องพิกเซลที่อยู่ภายในผลไม้ได้ทันที



8,500 Pixels

จำนวนพิกเซล (Pixel Count) = พื้นที่ (Area)

Bounding Box: ตีกรอบวัดความกว้าง-ยาว



Height Annotation
(ขอบบนสุดถึงขอบล่าง):
ตรวจสอบความยาว
รวมของผลผลิต

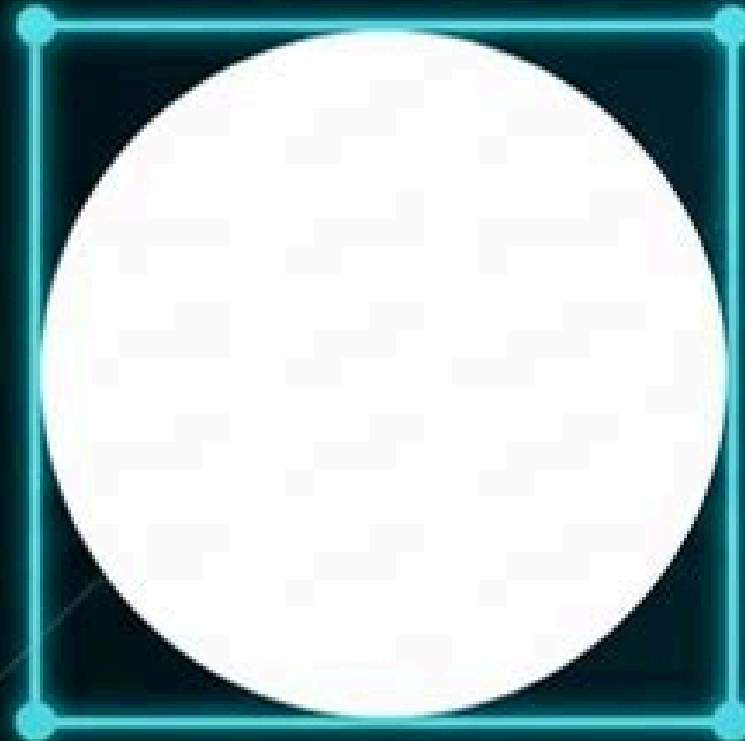
Width Annotation (ขอบซ้ายสุดถึงขอบขวา):
เช็กเส้นผ่านศูนย์กลางตามมาตรฐานตะแกรงโรงงาน

เกณฑ์คัดใช้มาตรฐาน (Standard Size Thresholds)

ขนาดเกรด	จำนวนพิกเซล (Area)	ความกว้างกรอบ (Width)	การจัดการหน้างาน (Destination)
● ไซส์ L	8,001 - 12,000 px	7.1 - 9.0 ซม.	คัดเกรดพรีเมียมส่งออกต่างประเทศ
● ไซส์ M	5,001 - 8,000 px	5.6 - 7.0 ซม.	ส่งซูเปอร์มาร์เก็ตชั้นนำ
● ไซส์ S	3,000 - 5,000 px	4.0 - 5.5 ซม.	ส่งกล่องตลาดทั่วไป / แปรรูป
● ไม่ได้ขนาด / ปิดตก	< 3,000 px	< 4.0 ซม.	ส่งปิดตกสายพานคัดทิ้งทันที

ขั้นตอนที่ 3: ขนาดเท่ากัน ไม่ได้แปลว่ารูปร่างสมบูรณ์

หากผลไม้มีพื้นที่ (Area) และกรอบสี่เหลี่ยม (Bounding Box) เท่ากัน
คอมพิวเตอร์จะแยก "ทรงกลม" กับ "ทรงเบี้ยว" หรือ "ขอบแหว่ง" ได้อย่างไร?



Area & Box = Same

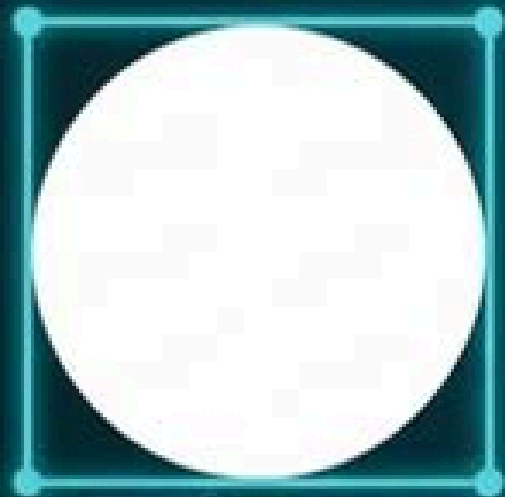


Area & Box = Same

↕ เทคนิคที่ 1: Aspect Ratio (สัดส่วนกว้าง-ยาว)

นำ ความกว้าง ÷ ความยาว

ทรงกลมที่สมบูรณ์



$$W \div L = 1.0$$



เข้าใกล้ 1.0 = ทรงกลมดี

ทรงยาว หรือ บิดเบี้ยว



$$W \div L < 0.8$$

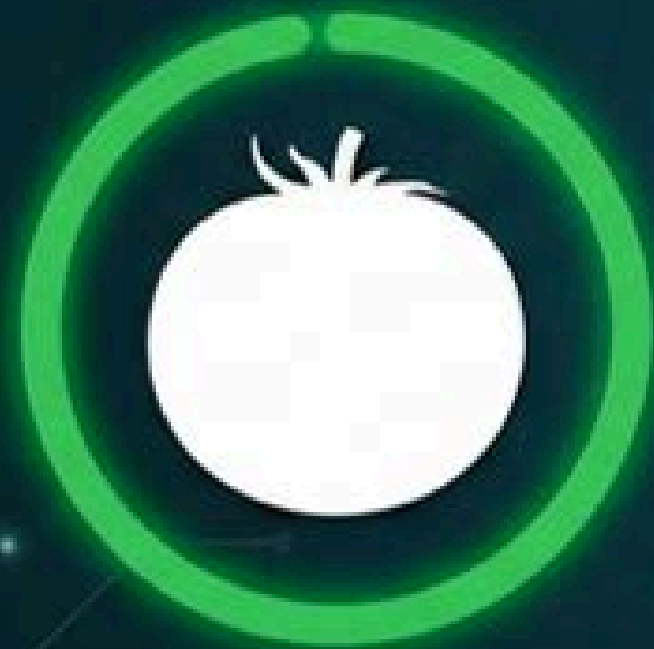


น้อยกว่า 0.8 = ทรงยาว หรือ บิดเบี้ยว

เทคนิคที่ 2: Roundness / Circularity (ความกลม)

วัดความสมบูรณ์ของวงกลม (เทียบพื้นที่จริงกับวงกลมสมมติ)

ทรงกลมที่สมบูรณ์



เข้าใกล้ 100% = ทรงกลมสวยงาม

ทรงเบี้ยว หรือ ผิดรูป



ต่ำกว่า 70% = ผลไม้บิดเบี้ยวผิดรูป



เทคนิคที่ 3: Perimeter (ความยาวขอบ)

วัดความยาวเส้นรอบรูปทั้งหมด

ผิวเรียบเนียน



✓ ผิวเรียบเนียน = ความยาวขอบปกติ

รอยเว้าแหว่ง / โดนแมลงกัด

⚠ Perimeter: HIGH



✗ รอยขรุขระทำให้ความยาวขอบสูงผิดปกติ

Shape Diagnostic Dashboard (สรุปเกณฑ์ตรวจสอบรูปทรง)



Aspect Ratio

เป้าหมาย: ~1.0

ปัญหาที่ตรวจจับ:
ทรงยาว/เบี้ยว



Roundness

เป้าหมาย: ~100%

ปัญหาที่ตรวจจับ:
การบิดเบี้ยวผิดรูปภาพรวม



Perimeter

เป้าหมาย: ปกติ (Smooth)

ปัญหาที่ตรวจจับ:
รอยเว้าแหว่ง/แมลงกัดกิน

Checklist หน้าที่งาน: สิ่งที่ต้องระวังเพื่อให้ระบบทำงานแม่นยำ



เทียบขนาดของจริง (Calibration)

วางวัตถุอ้างอิง (เหรียญ/แผ่นวัด)
เพื่อแปลงพิกเซลเป็นเซนติเมตร



ระยะห่างคงที่ (Fixed Distance)

ห้ามปรับความสูงกล้องขึ้นลง
เพราะขนาดพิกเซลจะหลุดตา



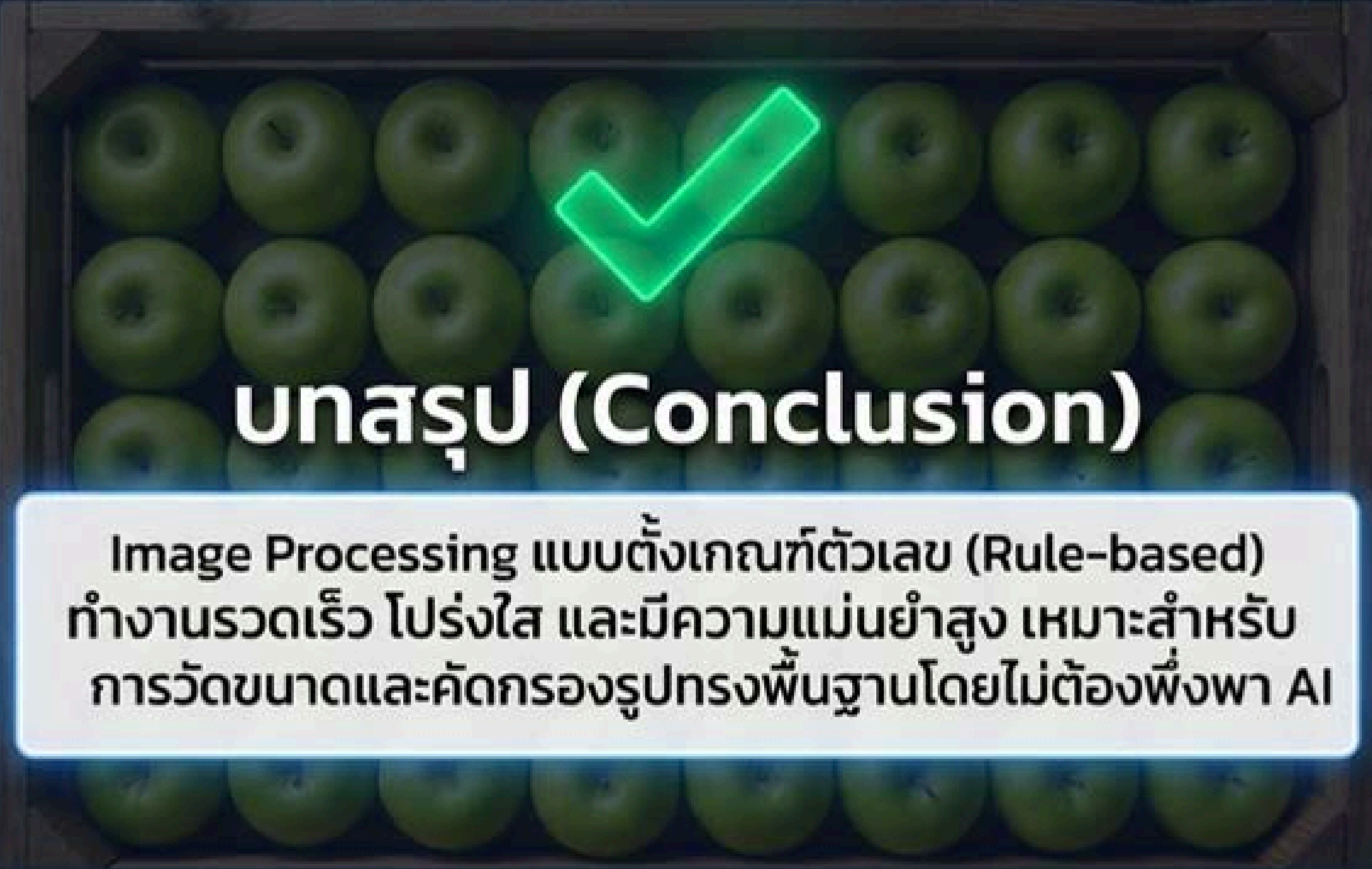
อย่าให้ผลไม้ติดกัน (No Touching)

คอมพิวเตอร์จะนับวัตถุที่แตะกัน
เป็น 1 ลูกไซส์ใหญ่ยักษ์



ทิศทางการวาง (Orientation)

ผลไม้เอียงทำให้กรอบ Bounding
Box กว้างผิดปกติ (แก้ไขด้วย
Rotate Bounding Box)

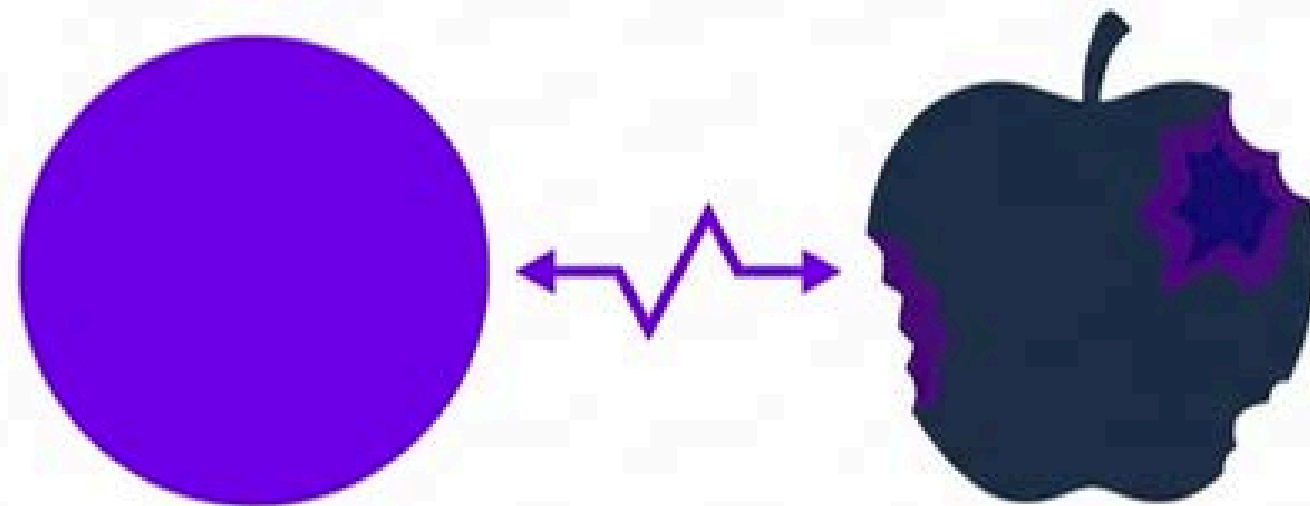


บทสรุป (Conclusion)

Image Processing แบบตั้งเกณฑ์ตัวเลข (Rule-based)
ทำงานรวดเร็ว โปร่งใส และมีความแม่นยำสูง เหมาะสำหรับการ
การวัดขนาดและคัดกรองรูปทรงพื้นฐานโดยไม่ต้องพึ่งพา AI

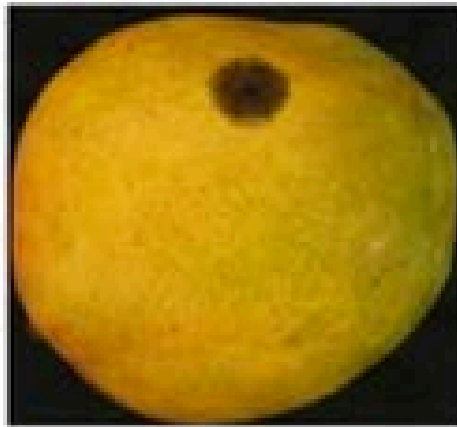
พัก 10 นาที ก่อนเริ่ม Session 3: AI กับการตรวจจับตำหนิผิวช้บซ้อน & เวิร์กช็อปจริง

ส่วนที่ 1 : ทำไมต้องใช้ AI?



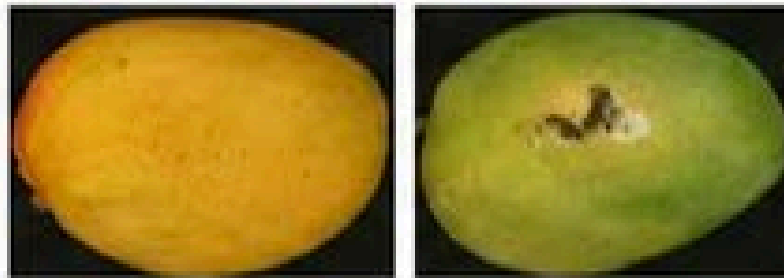
เมื่อรอยโรค รอยชำ และบาดแผลบนผลผลิต
มีรูปร่างและขนาดที่ไม่แน่นอน

การตรวจจับตำหนิซับซ้อนด้วย AI



จับรอยโรคและจุดชำ

แยกแยะจุดรอยโรคแคงเกอร์
รูหนอนเจาะ
หรือแผลรอยถูไถแม่นยำ



ไม่โดนลายธรรมชาติหลอก

แยกความต่างระหว่าง
"ลายผิวผลไม้ปกติ" กับ
"แผลเน่า" ได้อย่างถูกต้อง

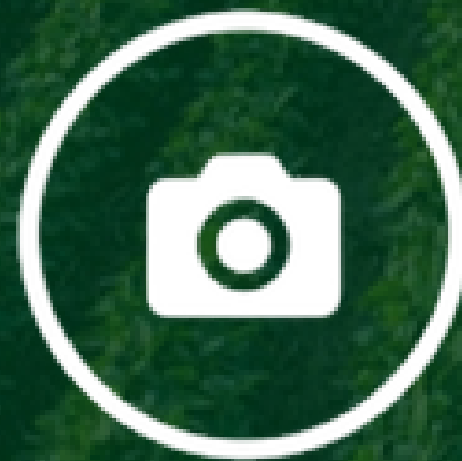


ยืดหยุ่นสูง

เมื่อเจอโรคพืชชนิดใหม่
เพียงถ่ายรูปเพิ่มเข้าไปสอน
AI เพิ่มเติมได้ทันที

ส่วนที่ 2: 3 ขั้นตอนสร้าง AI คัดเกรดผลผลิต

(เปลี่ยนจากการเขียนโค้ด มาเป็นการถ่ายภาพเพื่อสอนระบบ)



1. เก็บภาพตัวอย่าง
(Data)



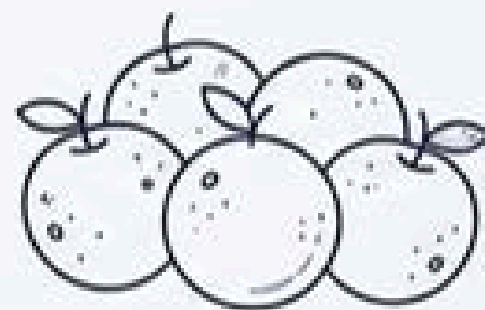
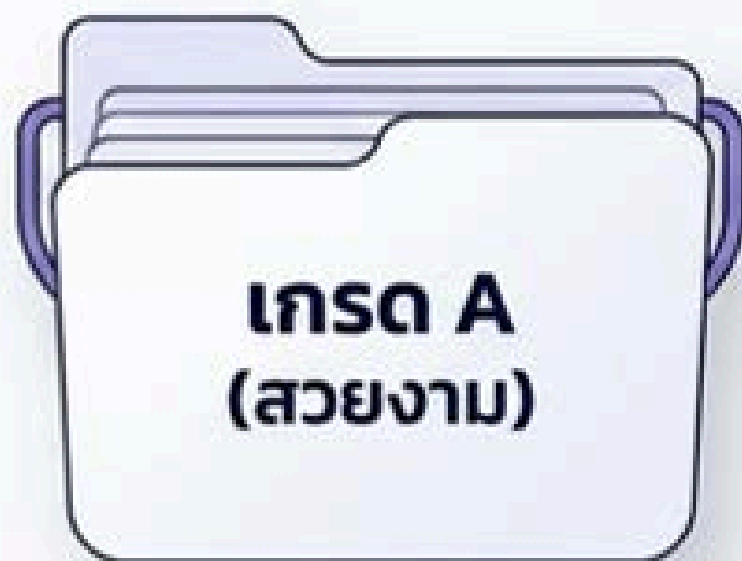
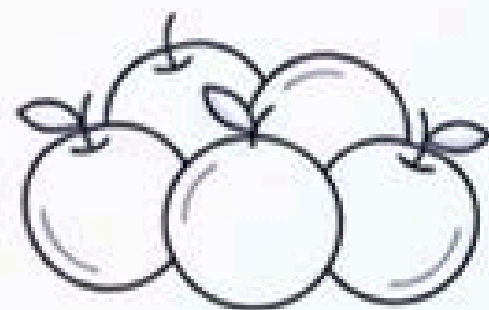
2. สอนโมเดล
(Train)



3. ทดสอบใช้งาน
(Test)

Step 1 : เก็บภาพตัวอย่าง (Data)

ถ่ายรูปผลไม้จริงแบ่งเป็นกลุ่ม กลุ่มละ 30-50 รูป:

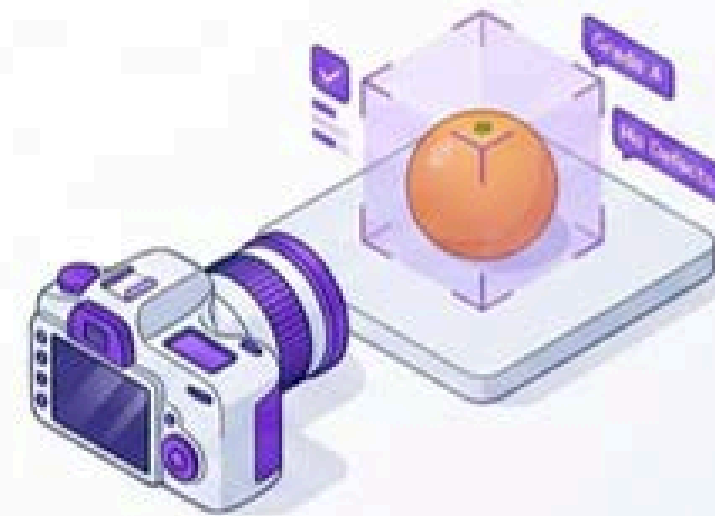


Step 2: สอนโมเดล (Train)



กดปุ่มส่งรูปเข้าโปรแกรม
ระบบจะคำนวณเปรียบเทียบหาจุดแตกต่าง
ของแต่ละกลุ่มให้อัตโนมัติในไม่กี่นาที

Step 3: ทดสอบใช้งาน (Test)



เปิดกล้องส่องผลไม้ลูกใหม่ที่ไม่เคยเห็นมาก่อน
เพื่อทดสอบว่า AI ทายเกรดและตรวจพบ
ตำหนิได้แม่นยำหรือไม่

เลือกใช้เครื่องมืออย่างไรให้เหมาะกับงาน?

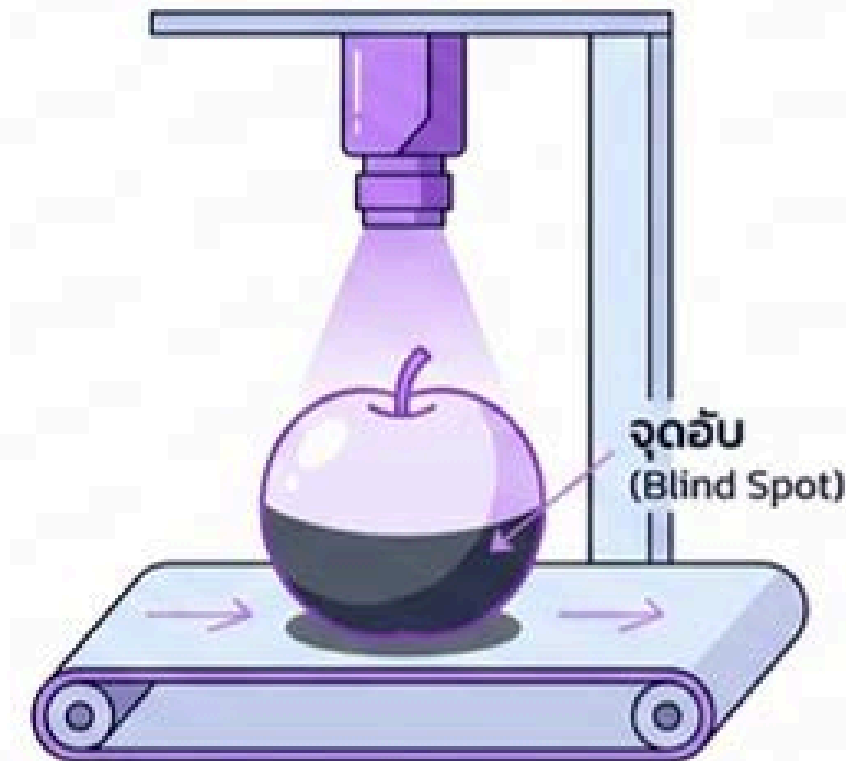
ประเภทงาน	เทคโนโลยี	จุดเด่น	ข้อแนะนำ
วัดขนาด / คัดไซส์ (S, M, L) & เช็กรูปทรง	Image Processing	ประมวลผลเร็วมาก แม่นยำ 100% / กินสเปกต่ำ	ใช้การนับพิกเซล & อัตราส่วน กว้าง ÷ ยาว
ตรวจรอยโรค / รอยชำ / แผล	AI (Machine Learning)	จัดการความซับซ้อน ของแผลได้ดี	ต้องถ่ายภาพตัวอย่าง แผลให้ครอบคลุม
คัดเกรดพรีเมียมส่งออก	Hybrid (ใช้คู่กัน)	ได้ทั้งขนาดตรง และผิวสวยไร้โรค	ใช้ Image Processing วัดไซส์ + AI เช็กผิว

ส่วนที่ 3 : ข้อควรระวังหน้างานจริง (ปัจจัยความสำเร็จในการติดตั้ง)



1. ระวังมุมอับของกล้อง

กล้องมองเห็นได้แค่ด้านบน
หากแพอยู่ด้านล่างจะมองไม่เห็น
ต้องใช้ลูกกลิ้งหมุนผลผลิตขณะ
วิ่งผ่านกล้อง





2. การดูแลเลนส์และฝุ่น

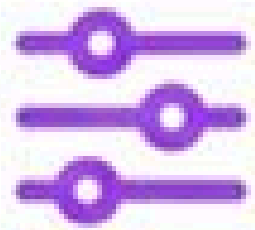
หน้างานโรงงานมีฝุ่น ละอองน้ำ
และคราบยางผลไม้ ต้องมีฝา
ครอบกล้องและทำความสะอาด
เลนส์ตามรอบ



3. ตัวอย่างรูปต้องหลากหลาย

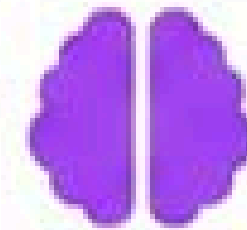
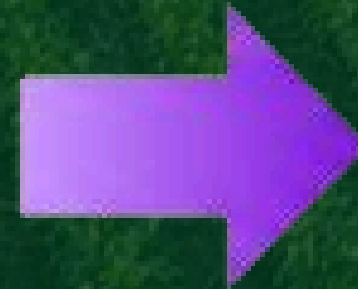
ควรถ่ายรูปผลไม้ในหลายๆ มุม
แสงหลายๆ แบบ เพื่อให้ AI
ฉลาดและไม่สับสนเมื่อเจอ
ผลผลิตลูกจริง

สรุปสูตรสำเร็จ : ระบบคัดแยกคุณภาพยุคใหม่



ด้านที่ 1: Image Processing

กรองจากหลัง ตัดขนาด และนับจำนวน
ผลผลิตได้อย่างรวดเร็ว



ด้านที่ 2: AI (Machine Learning)

ส่องวิเคราะห์คุณภาพผิว ตรวจจับรอยโรค
และการตัดเกรดพรีเมียม



รากฐาน: No-Code & Setup

เริ่มต้นจากการเข้าใจหลักการ ใช้เครื่องมือ No-Code ทดลอง และวางโครงสร้างกล่อง/แสงให้ถูกต้อง